

מטלה 3

אלגוריתמי פירוק ערך: IDQN, VDN ו-QMIX

הגשה בקבוצות של עד 3 סטודנטים

מטרת המטלה

במטלה זו נביר את IDQN — ההרחבה מרובת הסוכנים של DQN, שבה כל סוכן מאמן רשת Q עצמאית — ואת אלגוריתמי פירוק הערך VDN ו-QMIX. אלגוריתמים אלה מרחיבים את IDQN למשימות שבהן התגמול משותף לכל הסוכנים, ומפרקים את פונקציית ערך-הפעולה המשותפת לפונקציות תועלת פרטניות לכל סוכן. נבחן את תוצאותיהם של שלושת האלגוריתמים במשימת foraging level-based שיתופית ובשני תרחישים של סביבת SMAClite.

- IDQN — כל סוכן מאמן רשת DQN משלו ומתייחס לשאר הסוכנים כחלק מהסביבה
- VDN — ערך-הפעולה המשותף נאמד כסכום פונקציות התועלת של הסוכנים (פירוק אדיטיבי)
- QMIX — הכללה של VDN: רשת מערבבת (network mixing) המותנית במצב מחשבת שילוב

מונטוני של פונקציות התועלת

הערה: המטלה מבוססת על אותו codebase של מטלה 2 (marl-book-codebase). אם טרם התקנתם אותו, חזרו לשלבי ההכנה שם.

הכנת הסביבה

1. ודאו שה-codebase של הספר וסביבת lb-foraging מותקנים כפי שנעשה במטלה 2 — אתחול ה-submodule, התקנת התלויות והתקנת החבילה עצמה.
2. התקינו את סביבת SMAClite:

```
pip install git+https://github.com/uoae-agents/smaclite.git
```

3. בדקו שהכול עובד: נווטו לתיקייה marlbase והריצו אימון קצר של IDQN; אמורה להופיע התקדמות אימון סדירה בקונסול:

```
python run.py +algorithm=idqn env.name="lbforaging:Foraging-8x8-2p-3f-v3"
env.time_limit=50 algorithm.total_steps=100000
env.wrappers="[CooperativeReward]"
```

פירוק ערך במשימת Foraging Level-Based

נתמקד במשימה Foraging-8x8-2p-3f. כדי לאמן את IDQN עם תגמול משותף נשתמש ב-wrapper בשם CooperativeReward, המעניק לכל הסוכנים תגמול זהה השווה לסכום התגמולים האינדיבידואליים. אימון IDQN בתצורה זו:

```
python run.py +algorithm=idqn env.name="lbforaging:Foraging-8x8-2p-3f-v3"
env.time_limit=50 algorithm.total_steps=4000000 algorithm.eval_interval=100000
algorithm.log_interval=100000 env.standardise_rewards=True
env.wrappers="[CooperativeReward]"
```

אימון VDN ו-QMIX נעשה באותה פקודה, תוך החלפת הארגומנט `algorithm+ vdn` או `qmix`. במקרים אלה אין צורך לציין את ה-`wrapper` במפורש, שכן תצורות ה-VDN וה-QMIX כבר מגדירות את תצורת התגמול המשותף.

אימון שלוש האלגוריתמים במשימה זו אורך זמן רב, ולכן השתמשו בנתוני האימון המוכנים שבתיקייה `deep_marl_data/lbf_8x8-2p-3f_coop`, הכוללים ריצות של IDQN, VDN ו-QMIX. השוו את ביצועי האלגוריתמים ובחנו את התשואות ומדדים נוספים:

```
python utils/postprocessing/plot_runs.py --source
path/to/deep_marl_data/lbf_8x8-2p-3f_coop --metric ...
```

כדי להתרשם מהמדיניות הנלמדת, צפו בסרטוני ה-rollout השמורים בתיקיית `videos` של כל ריצה, או הפיקו סרטון מתוך `checkpoint` באמצעות סקריפט ההערכה, כפי שעשינו במטלה 2:

```
python eval.py path=path/to/deep_marl_data/lbf_8x8-2p-3f_coop/...
env.name="lbforaging:Foraging-8x8-2p-3f-v3" env.time_limit=50
```

פירוק ערך בסביבת SMAClite

לסיום נעבור לסביבה שיתופית לחלוטין: SMAClite, המשחזרת בקירוב את SMAC (Multi-StarCraft Challenge Agent) — סביבה נפוצה למשימות שיתופיות לחלוטין. בתרחישי SMAC כל סוכן שולט ביחידה בודדת בקבוצה, ועל הקבוצה להכריע קבוצת יחידות אויב המנוהלת על ידי AI מובנה. כל הסוכנים מקבלים תגמול זהה ורואים מידע חלקי בלבד.

לדוגמה, אימון IDQN בתרחיש `2s3z` של SMAClite:

```
python run.py +algorithm=idqn env.name="smac_lite/2s3z-v0" env.time_limit=150
algorithm.total_steps=1000000 algorithm.eval_interval=1000
algorithm.log_interval=1000 env.standardise_rewards=True
```

SMAClite כבדה חישובית משמעותית ממשימות ה-foraging level-based, וזמני האימון בה ארוכים יותר. בתיקיית `deep_marl_data` מסופקים נתוני אימון של IDQN, VDN ו-QMIX בשני תרחישים: `2s3z` ו-`2s_vs_1sc`. השוו את ביצועי האלגוריתמים בשני התרחישים, בחנו את מדדי האימון כפי שעשיתם קודם וצפו בסרטוני ה-rollout. נסו לזהות התנהגויות מעניינות המתפתחות במהלך האימון — למשל תנועה מושכלת של יחידות טווח לעומת יחידות קרב-פנים, או תקיפה מתואמת של יחידות אויב.

הערה: כמו במטלה 2, בריצות שבתיקיית `deep_marl_data` בוצע כוונן היפר-פרמטרים מינימלי בלבד, ולכן הביצועים והדירוג היחסי בין האלגוריתמים אינם בהכרח משקפים את יכולותיהם תחת כוונן מלא.

Assignment 3

Value Decomposition Algorithms: IDQN, VDN and QMIX

Submission in groups of up to 3 students

Goal

In this assignment we look at independent DQN (IDQN), the multi-agent extension of DQN in which each agent trains its own Q-network, and at the value decomposition algorithms VDN and QMIX. These algorithms extend IDQN to common-reward multi-agent tasks and decompose the joint action-value function into individual utility functions for each agent. We will inspect the results of all three algorithms in a cooperative level-based foraging task and in two scenarios of the SMAclite environment.

- IDQN — each agent trains its own DQN and treats the other agents as part of the environment
- VDN — the joint action-value is estimated as the sum of the individual utilities of the agents (additive decomposition)
- QMIX — a generalisation of VDN: a state-conditioned mixing network combines the individual utilities monotonically

Note: this assignment builds on the same codebase as Assignment 2 (marl-book-codebase).

If you have not installed it yet, go back to the setup steps there.

Setup

1. Make sure the marl-book codebase and the level-based foraging environment are installed as in Assignment 2 — submodule initialisation, dependencies, and the codebase itself.
2. Install the SMAclite environment:

```
pip install git+https://github.com/uoε-agents/smaclite.git
```

3. Test that everything works: navigate to the marlbase directory and run a short IDQN training run; you should see regular training progress printed to the console:

```
python run.py +algorithm=idqn env.name="lbforaging:Foraging-8x8-2p-3f-v3"  
env.time_limit=50 algorithm.total_steps=100000  
env.wrappers="[CooperativeReward]"
```

Value Decomposition in Level-Based Foraging

We focus on the Foraging-8x8-2p-3f task. To train IDQN with common rewards we use the CooperativeReward wrapper, which gives every agent the same reward, defined as the sum of all individual agents' rewards. Training IDQN in this setup:

```
python run.py +algorithm=idqn env.name="lbforaging:Foraging-8x8-2p-3f-v3"  
env.time_limit=50 algorithm.total_steps=4000000 algorithm.eval_interval=100000  
algorithm.log_interval=100000 env.standardise_rewards=True  
env.wrappers="[CooperativeReward]"
```

VDN and QMIX are trained with the same command, substituting the +algorithm argument with vdn or qmix. In these cases the wrapper does not need to be specified explicitly, since the VDN and QMIX configurations already specify this common-reward setup.

Training all three algorithms in this task takes considerable time, so use the training data already prepared in deep_marl_data/lbf_8x8-2p-3f_coop, covering IDQN, VDN and QMIX.

Compare the algorithms and inspect the episodic returns and other metrics:

```
python utils/postprocessing/plot_runs.py --source  
path/to/deep_marl_data/lbf_8x8-2p-3f_coop --metric ...
```

To get a sense of the learned policies, watch the rollout videos stored in the videos directory of each run, or generate a video from a model checkpoint with the evaluation script, as in Assignment 2:

```
python eval.py path=path/to/deep_marl_data/lbf_8x8-2p-3f_coop/...  
env.name="lbforaging:Foraging-8x8-2p-3f-v3" env.time_limit=50
```

Value Decomposition in SMAclite

Finally, we turn to the fully cooperative SMAclite environment, which closely simulates the StarCraft multi-agent challenge (SMAC), a commonly used environment for fully cooperative tasks. In SMAC scenarios each agent controls a single unit of a team, and the team must defeat an enemy team of units controlled by a built-in AI. All agents receive identical rewards and observe only partial information.

For example, an IDQN training run in the 2s3z scenario of SMAclite:

```
python run.py +algorithm=idqn env.name="smaclite/2s3z-v0" env.time_limit=150  
algorithm.total_steps=1000000 algorithm.eval_interval=1000  
algorithm.log_interval=1000 env.standardise_rewards=True
```

SMAClite is significantly more computationally expensive than the level-based foraging tasks, and training runs take longer to complete. The `deep_marl_data` directory provides training data of IDQN, VDN and QMIX in two SMAClite scenarios: `2s3z` and `2s_vs_1sc`. Compare the performance of the algorithms in both scenarios, inspect the training metrics as before, and watch the rollout videos. Can you spot interesting behaviours of the agents as training progresses, such as strategic movement of ranged and melee units, or coordinated attacks on enemy units?

Note: as in Assignment 2, only minimal hyperparameter tuning was performed for the training runs in `deep_marl_data`, so the performance and relative ranking of the algorithms might not be reflective of their capabilities under tuned hyperparameters.